

ProjWT — Deep-Dive & Interview Masterclass

Purpose: everything you need to explain, defend, and extend this project in depth. Read it end to end once; skim sections before an interview. Where it says **"say this,"** that's a ready answer. Where it says **"why,"** that's the reasoning that impresses.

The project in one sentence: **"I reverse-engineered the Bluetooth protocol of a WHOOP 4.0 fitness band and built a fail-safe gesture-control system that commands a real Tesla through its signed, production vehicle API — you tap the band, it buzzes back, and the car responds."**

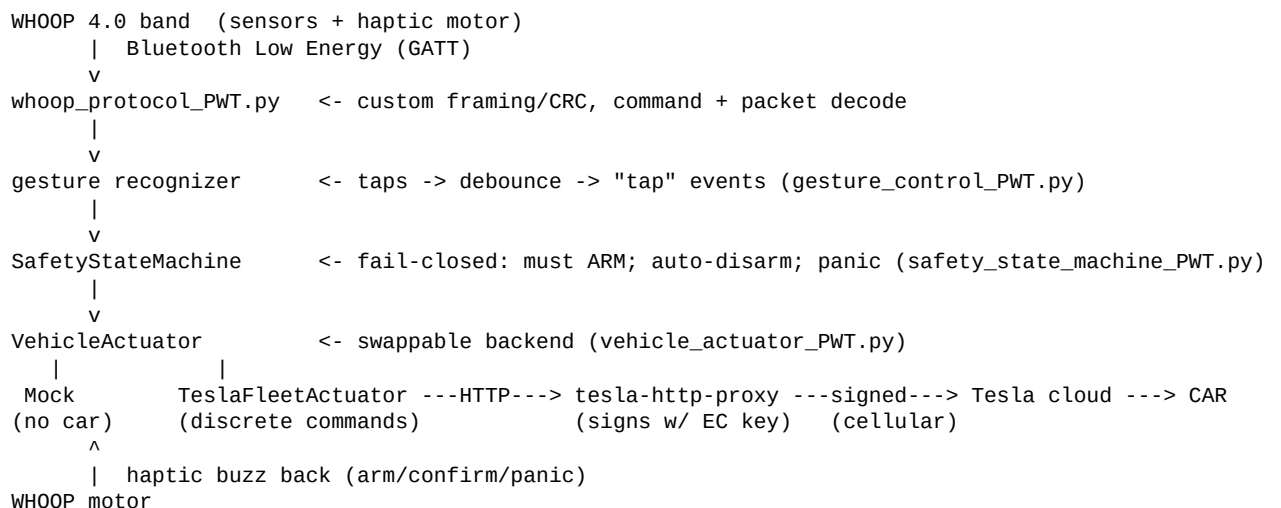
PART 1 — THE 30-SECOND / 2-MINUTE / 10-MINUTE PITCHES

30 seconds: "ProjWT is a wearable gesture controller. I reverse-engineered the undocumented BLE protocol of my WHOOP band so I can read its sensors and taps, then built a pipeline that turns a tap pattern into a signed command to my Tesla — unlock, climate, charging — with a fail-closed safety layer and haptic feedback. It unlocks my real car when I tap my wrist."

2 minutes: add the layers — "The band speaks a custom GATT protocol I had to reverse-engineer: framing, CRC, command opcodes. On top of that I decode live heart rate and pull raw banked sensor data. Taps go through a recognizer I had to design around the hardware's quirks, then a fail-closed state machine that requires arming, then a swappable 'actuator' layer. For the car, modern Teslas require every command to be cryptographically signed — so I set up OAuth, hosted a signing key, paired a virtual key to the car, and run Tesla's signing proxy locally. The whole thing launches with one command."

10 minutes: walk the architecture diagram (Part 2), then pick two deep dives — the BLE reverse-engineering (Part 4) and the command signing (Part 9) are the most impressive. Finish with a debugging war story (Part 12).

PART 2 — ARCHITECTURE (the whole system)



Why layered this way: each layer has one job and a clean interface, so any piece can be swapped or tested alone. The actuator is the key abstraction — the exact same gesture→safety pipeline drives a Mock (for testing), the real Tesla, or (future) a simulator, by changing only the backend. ***That's the design**

decision an interviewer will respect: dependency inversion — the policy (gestures/safety) doesn't depend on the mechanism (which car/sim).*

Data flows two ways: commands flow down (tap → car); feedback flows back up (the band buzzes to confirm). Biometrics flow sideways (band → dashboard).

PART 3 — THE TECH STACK & WHY

Choice	Why	Why not the alternative
Python (Phase 1)	Fastest to prototype BLE + protocol RE; bLeak is excellent	C++/UE5 is the eventual target, but you don't reverse-engineer a protocol in C++ — you prototype fast, then port
bLeak (BLE lib)	Cross-platform, async, works on the Windows WinRT backend	Native WinRT directly = far more code for the same thing
Pure-Python deps only (reportlab, python-docx, urllib)	Zero heavyweight installs; runs anywhere	LibreOffice for doc export *failed in the sandbox* — learned to avoid external binaries
Fail-closed state machine	Safety-critical: the default state is STOPPED	A flag/boolean would miss edge cases (dropouts, stale heartbeats)
Local signing proxy	Private key never leaves the machine	A cloud signer is convenient but puts the key on a server (bigger attack surface)
Tap-dwell UX	Robust to the hardware's tap bounce	Counting taps misfired constantly (see Part 8)

PART 4 — BLE / GATT & THE WHOOP PROTOCOL (the reverse-engineering)

Background to say out loud: "BLE devices expose *services*, each containing *characteristics* — think of a characteristic as a variable you can read, write, or *subscribe to* for notifications. The WHOOP has a standard Heart-Rate service and a Battery service, plus a **custom vendor service** that carries everything interesting, and that custom service is completely undocumented — that's what I reverse-engineered."

The map I built:

- Custom service: 61080001-8d6d-82b8-614a-1c8cb0f8dcc6
- ...02 = **command write** (I send commands here)
- ...03 = **command response** (RESP)
- ...04 = **events** (taps/motion, EVENT)
- ...05 = **data** (HR, historical records, DATA)

- . . . 07 = **diagnostics** (DIAG)
- Standard 0x2A37 = Heart Rate, 0x2A19 = Battery Level

Gotcha worth mentioning: every custom characteristic UUID **ends** the same (. . . b0f8dcc6) and differs only in the **first** block — so you must label them by the leading bytes, never the trailing ones. (Small detail that shows you actually read the bytes.)

The packet framing (the core RE win)

Every command/packet is wrapped in a frame I had to deduce:

```
0xAA | length(2 bytes, little-endian) | CRC8(of the length) | [type][seq][cmd][payload] | CRC32(4 bytes, little-endian)
```

- 0xAA = start-of-frame marker.
- Length = size of the inner record + 4 (for the CRC32).
- **CRC8** over the length bytes, polynomial 0x07, no reflection — a header checksum.
- **inner record** = [packet type][sequence][command/subtype][payload . . .].
- **CRC32** (the zlib/Ethernet variety) over the inner record — a body checksum.

How I verified it (the credible part): I didn't guess and hope. I captured real packets, and I cross-checked the framing against an independent open-source implementation (tanarchytan/whoop-rs, MIT-licensed) — the CRC8 polynomial, the CRC32 variety, and the field layout matched **byte-for-byte**. Two independent derivations agreeing is strong evidence it's correct. *Say: "I validated my reverse-engineering against a second independent implementation and real captured packets — not by guessing."*

Packet types (inner byte 0) — verified values

Value	Name	Meaning
0x23 (35)	COMMAND	outbound command
0x24 (36)	COMMAND_RESPONSE	ack/result
0x28 (40)	REALTIME_DATA	live HR/RR
0x2b (43)	REALTIME_RAW_DATA	some live raw frames
0x2f (47)	HISTORICAL_DATA	banked records
0x30 (48)	EVENT	taps/motion
0x31 (49)	METADATA	history start/end/complete
0x32 (50)	CONSOLE_LOGS	firmware debug strings
0x33 (51)	REALTIME_IMU	100 Hz IMU (Gen5 only)

A real bug I found and fixed: I had HISTORICAL mislabeled as 0x32; the true value is 0x2f, and 0x32 is actually console logs. Finding this came from cross-checking the reference — and it mattered because the historical decoder keyed off that type byte.

Command opcodes (inner byte 2)

HELLO 0x05, TOGGLE_REALTIME_HR 0x03 (payload 0x01 = start), STOP 0x04, SEND_HISTORICAL 0x16, HISTORICAL_RESULT 0x17, ABORT_HISTORICAL 0x14, SEND_OPTICAL 0x6b, SET_IMU_STREAM 0x6a, RUN_HAPTICS 0x4f (pattern 2 = the buzz).

`build_packet()` *and* `unframe()` — *line by line*

```
def crc8(data):                                # header checksum, poly 0x07
    crc = 0
    for b in data:
        crc ^= b
        for _ in range(8):
            crc = ((crc << 1) ^ 0x07) & 0xFF if crc & 0x80 else (crc << 1) & 0xFF
    return crc
```

Standard bit-by-bit CRC: XOR the byte in, then for each of 8 bits, shift left and conditionally XOR the polynomial if the top bit was set, masking to 8 bits.

```
def build_packet(cmd, seq=0, data=b"", pkt_type=0x23):
    payload = bytes([pkt_type, seq, cmd]) + data # the inner record
    body = payload + crc32_le(payload)           # append body CRC32
    header = bytes([0xAA]) + len(body).to_bytes(2, "little")
    return header + bytes([crc8(header[1:3])]) + body
```

Build the inner record, append its CRC32, prepend the SOF + length + length-CRC8. `unframe()` reverses it: strip the header, verify CRCs, return the inner record.

PART 5 — PAIRING & THE WINDOWS BLE QUIRKS

- The command characteristic needs an **encrypted link**, so you must **bond** (pair) first. On Windows I use the WinRT "just works"/ConfirmOnly pairing ceremony with encryption.
- **Idle-drop**: the band drops the BLE link after ~1s of silence. Fix: a periodic harmless **battery read** as a keepalive — it holds the connection without being a WHOOP command.
- **"The operation was canceled by the user" (WinError -2147023673)**: a WinRT GATT flakiness. Fix: a reconnect loop with backoff; sometimes needs a Bluetooth off/on.
- **"Tap the band to wake it"**: the accelerometer sleeps; see Part 8.

Interview angle: "Working with real hardware means handling the messy reality — connection drops, power management, OS-specific BLE stacks. I built keepalives and reconnect logic rather than assuming a perfect link."

PART 6 — LIVE BIOMETRICS (heart rate)

- Send TOGGLE_REALTIME_HR with payload 0x01 → the band streams **type 0x28** packets on the DATA characteristic at ~1 Hz.
- Decode: timestamp (uint32 LE), HR (a byte), then R-R intervals (uint16 LE, the millisecond gaps between heartbeats — the basis of heart-rate variability).
- **Validated** against the actual BPM shown — the decode is correct, not just plausible.

PART 7 — RAW SENSOR DATA (the Gen4 reality)

This is a great "I dug deep and found the truth" story.

The goal: live raw IMU (accelerometer + gyroscope) off the band. **The finding:** the 100 Hz raw IMU stream (record "v21") and raw optical buffer ("v20") are **Gen5 / WHOOP 5.0 only** — they ride the newer "R22 deep buffers." A WHOOP 4.0 (Gen4) does not expose a live IMU firehose. That's a hardware/firmware boundary, not a bug.

But Gen4 does bank rich data — and I was fetching it wrong at first. Gen4 records ("v24"/"v12"): heart rate, R-R, **gravity vector** (orientation, from the accelerometer), SpO2 (raw red/IR), skin temperature, respiration. They come over the **historical offload** path, which is a **sequence**, not a single command:

- 1 SEND_OPTICAL [01, 01] — enable collection (no ack is expected).
- 2 SEND_HISTORICAL [00] — **kick the drain** (the step I was missing — nothing came until I added it).
- 1 For each METADATA HistoryEnd, write HISTORICAL_RESULT [01]+end_data to ack the chunk so the band advances instead of stalling.
- 1 Stop on HistoryComplete; turn collection off.

I drained **2,429 per-second records** (~40 min) and decoded HR 42–80 bpm plus clean gravity vectors (99% within 0.9–1.1 g after a fix).

A precise bug to tell: for v24 records the gravity vector is three float32 at offsets 36/40/44 — I first read it as int16, which produced garbage magnitudes ($|g|$ 1.86 instead of ~1.0). Reading the exact field type from the reference fixed it to $|g|=0.995$. **This shows you can debug at the byte level.**

Why "banked, not live" matters: offload data is seconds old — fine for a dashboard or coarse orientation, wrong for a fast control loop. Which is exactly why the continuous-hold **driving** idea uses the **phone IMU**, not the band (Part 11).

PART 8 — GESTURE RECOGNITION (the UX engineering)

The most **iterated** part — great to discuss because it shows user-centered debugging.

Problem 1 — flicks needed brute force. The band only reports motion while its accelerometer is **awake**, and the accelerometer uses firmware **wake-on-motion** power management I can't override over BLE. Proof: I streamed HR continuously (sensor on) and still only one motion event fired — on a tap. So keeping HR on does **not** keep motion sampling awake.

Fix 1 — tap, don't flick. A finger-tap on the band face delivers a sharp localized impulse (same 0x0e events) with far less wrist strain than flinging the arm. First tap is firmer (wakes the accelerometer); then light taps register. Calibration on my wrist: noise < ~1600, intentional light taps ~2400+, so the threshold sits at 2200.

Problem 2 — counting taps misfired. I first classified a burst by count: 1=cycle, 2=confirm, 3+=panic. But taps **bounce** — a double-tap's ringing added a phantom third tap → read as a panic. Timing alone can't separate bounce from an intentional second tap (they're ~0.3 s apart either way).

Fix 2 — tap-to-cycle, pause-to-confirm (dwell). Each debounced tap advances a menu; **pausing ~2.5 s on a command executes it.** Bounce can only over-advance the menu (visible on screen, recoverable), never fire the wrong thing or panic. The menu ends in CANCEL so a dwell can safely back out. *This is the key UX insight: when the input is noisy, design an interaction where noise is harmless.*

The recognizer internals: a refractory window (- - flick-gap, 0.3 s) collapses one tap's ringing into a single "tap"; the dwell timer (- - dwell, 2.5 s) fires the selection. All tunable; - - calibrate prints strengths with no commands sent.

PART 9 — THE SAFETY STATE MACHINE (fail-closed)

Say: "Anything touching a vehicle has to be fail-safe, so I built a fail-closed state machine: the **default** state is stopped, and anything going wrong returns to stopped."

- States: IDLE (stopped, default) → ARMED (a deliberate gesture armed it) → HOLDING (active hold, for the future drive loop) → STOPPING (ramp to stop) → back to IDLE.
- **Fail-closed triggers:** arm timeout, stale heartbeat, lost hold, disarm/panic gesture, any fault → all route to stopping/idle.
- It's **pure logic, unit-tested** (7 tests) — no hardware needed to verify the safety rules. *That's a deliberate choice: isolate the safety-critical logic so it's testable.*
- In the discrete-command world it enforces: **no command fires unless armed**, it auto-disarms on inactivity, and the panic path locks + disarms.

Why this matters for the "can it drive the car" question (below): the fail-closed design is the only latency-robust way to do vehicle control — you never depend on a fast "STOP" reaching the car; absence of "keep going" **is** stop.

PART 10 — HAPTIC FEEDBACK (closing the loop)

The WHOOP has a buzz motor (for silent alarms). I drive it via RUN_HAPTICS_PATTERN (opcode 0x4f, body [pattern, loops, 0, 0, 0], pattern 2 = the 4.0 buzz). The controller buzzes on connect / arm / confirm / panic, so you feel state without looking at a screen. *Interview angle: a control system with no feedback is frustrating; closing the loop is what makes it feel like a product.*

PART 11 — TESLA INTEGRATION (OAuth, scopes, signing) — the systems story

This is the densest, most "senior" part. Know it cold.

The API

Tesla's **Fleet API** is an OAuth2 + REST JSON API. With the owner's consent you can read vehicle data and send **discrete commands** (lock, climate, charging, horn, lights, trunk, windows...). **There is no third-party API for vehicle **motion** / Summon** — that lives only inside Tesla's own app behind proximity

+ continuous-press interlocks (Part 13).

OAuth 2.0 — two token types

- **Authorization Code flow** (user login): the owner logs in on Tesla's site, which redirects back to `localhost:3000/callback` with a code; I exchange the code (+ client secret) for an **access token** (~8 h) and a **refresh token**. The redirect URI is just where the one-time login sends the code; it's not used at runtime.
- **Client Credentials flow** (partner token): app-level, used once to register the domain. *Gotcha: it rejects `openid/offline_access` scopes — those are user-login only.*
- **Scopes** requested: `vehicle_device_data`, `vehicle_cmds`, `vehicle_charging_cmds` (plus `openid offline_access` for the user login's refresh token).

Command signing — the Vehicle Command Protocol (VCP)

Modern cars (incl. my 2018 Model 3) **refuse unsigned commands** ("Tesla Vehicle Command Protocol required"). Every actuation must be cryptographically signed. The chain:

- 1 I generated an **EC key pair** (NIST P-256 / prime256v1) with OpenSSL. Private key stays on my machine; public key is hosted at `https://<domain>/.well-known/appspecific/com.tesla.3p.public-key.pem`.
- 1 **Partner registration** (`/api/1/partner_accounts` with my domain) tells Tesla's backend to fetch that public key.
- 1 **Virtual-key pairing**: open `https://tesla.com/_ak/<domain>` on the owner's phone and approve — this adds my public key to the **car's** trust store.
- 1 At runtime, **tesla-http-proxy (Tesla's Go tool)** takes my **plain REST call**, signs the command body with my private key, negotiates a session with the car's `DOMAIN_VEHICLE_SECURITY` + `DOMAIN_INFOTAINMENT`, and forwards it. The car verifies the signature against the paired public key and executes.

Analogy that lands: "The key pair is a digital key fob I minted. The private key is the fob in my pocket that signs 'unlock.' The public key is the car's record of which fobs it trusts — pairing is adding my fob to the car."

Why run the proxy locally: the private key never leaves my machine. A hosted signer is more convenient (works from anywhere) but widens the attack surface — a tradeoff I can discuss (Part 14, future work).

The data-vs-command distinction

Reading `vehicle_data` only needs the bearer token (no signing). Only **commands** need signing. That's why the dashboard can read state directly but unlock must go through the proxy.

PART 12 — THE DEBUGGING WAR STORIES (gold for interviews)

Each of these is a "tell me about a hard bug" answer. Pick 2–3.

- 1 **The truncated client secret.** `unauthorized_client` on every token call, even in raw curl. The ID was verified correct. The cause: the Tesla dashboard has no copy button on the secret, and hand-selecting it dropped a character. *Lesson: isolate the variable — I proved it wasn't my code by reproducing the failure in a raw curl, then proved it wasn't quoting, then narrowed to the secret value itself.* **Method over guessing.**
 - 1 **GitHub Pages wouldn't serve the key.** The `.well-known` folder starts with a dot, and Jekyll (Pages' default) ignores dotfolders → the key 404'd silently. Fix: a `.nojekyll` file. *Lesson: know your platform's defaults.*
 - 1 `go install` refused the proxy ("go.mod has replace directives"). Fix: `clone + `go build``. *Lesson: when the convenient path fails, drop one level down.*
 - 1 **Git Bash mangled the TLS cert subject.** `-subj "/CN=localhost"` became a Windows path (`C:/Program Files/Git/CN=...`) via MSYS path conversion → the cert was never written → the proxy couldn't start. Fix: `MSYS_NO_PATHCONV=1`. *Lesson: cross-tooling on Windows has sharp edges.*
 - 1 **Wrong vehicle identifier.** The signing proxy wants the **VIN** in the path, not the numeric Fleet API ID (expected 17-character VIN). One-line fix, but you have to read the error.
 - 1 **Em-dash broke PowerShell.** Non-ASCII characters in a `.ps1` file broke the PS 5.1 parser. Fix: keep scripts pure ASCII. *Lesson: encoding matters.*
 - 1 The 0x2f vs 0x32 packet-type bug **and the** float32-vs-int16 gravity bug — both found by cross-checking the reference, both byte-level.

Meta-point to make: "Most of these weren't code bugs — they were integration reality: platform defaults, encoding, tooling, hardware power management. That's most of real engineering."
-

PART 13 — "WHY DIDN'T YOU..." (the decisions you'll be challenged on)

"Why not make the car actually drive / summon to you?" Two reasons, and lead with the hard one: *there is no third-party API for vehicle motion.* Smart/Actually/Dumb Summon live only inside Tesla's own app behind proximity + continuous-press + line-of-sight interlocks — zero programmatic hook for anyone. So it's not a matter of latency or effort; the endpoint doesn't exist. And second, deliberately bypassing a 2-ton vehicle's safety interlocks is out of scope — unsafe and against ToS. The continuous-hold *drive* demo is therefore built against a **simulator**, which also proves the control loop more cleanly.

"Why the phone IMU for the hold loop instead of the band?" Latency and exposure. The band's motion is coarse and, on Gen4, only available banked (seconds old). The phone IMU is 50–100 Hz at ~5–20 ms. And the real bottleneck is the **Tesla cloud round-trip (1–5 s)** — so the dead-man's-switch is designed to **fail safe** (absence of "keep going" = stop), never to rely on a fast stop command.

"Why Python if the target is C++/UE5?" You prototype a reverse-engineering effort in the fastest iteration loop you have, then port the proven design. Rewriting unknowns in C++ first would be slow and wasteful.

"Why discrete commands only?" It's what the API safely exposes, and it's a complete, honest demo of the gesture→safety→signed-API pipeline.

PART 14 — FUTURE WORK & WORKAROUNDS (shows vision)

- **Phone app (Flutter):** BLE to the band + command UI + dashboard, calling a signer.

The hard part is signing on mobile — options: (a) **host the proxy** on an always-on box behind authenticated HTTPS (fast; key on a server you secure), or (b) compile Tesla's Go signing into the app via gomobile (self-contained; more work).

- **"From anywhere" (hosted signer):** put `tesla-http-proxy` on a home server/VPS; then even the current code works from any parking lot with internet. Security: authenticate the endpoint so only my app can call it.
 - `climate_smart` as a tap: wire the body-temp→cabin feature to a menu entry.
 - **Continuous-hold drive demo:** 2D sim first, then UE5 — the original "summon" vision, done safely.
 - **BLE stability:** treat the WinRT cancel as a silent auto-reconnect for seamless demos.
 - **Why not done yet:** the app needs device/build tooling and store provisioning; hosting needs a server + hardening. These are deployment efforts, not unknowns — scoped, not blocked.
-

PART 15 — LIKELY INTERVIEW QUESTIONS (rapid-fire answers)

- **"Walk me through the architecture."** → Part 2 diagram; emphasize the swappable actuator (dependency inversion).
- **"How did you reverse-engineer the protocol?"** → captured packets + cross-checked an independent impl; deduced framing (SOF/length/CRC8/record/CRC32); validated byte-for-byte.
- **"How do you keep it safe?"** → fail-closed state machine, must-arm gate, auto-disarm, dwell confirm, and the fact that vehicle motion is intentionally out of scope.
- **"How does command signing work?"** → EC key pair, hosted public key, partner registration, virtual-key pairing, local proxy signs with VCP; car verifies.
- **"Hardest bug?"** → pick from Part 12 (the secret-truncation isolation story is the best — it shows method).
- **"What would you change / how would you scale?"** → Part 14; hosted signer + mobile app; mention securing the key.
- **"Security concerns?"** → private key stays local; secrets/tokens/MFA codes gitignored; a hosted signer must be authenticated; OAuth tokens expire + refresh.
- **"Did you use AI?"** → "Yes, to accelerate the implementation. I reverse-engineered the

protocol, designed the architecture, debugged the hardware, and integrated the signed API — ask me anything about how any of it works." Then answer deeply. *Owning it + demonstrating understanding beats hiding it every time.*

- **"What did you learn?"** → that most of real engineering is integration reality

(platforms, encoding, tooling, hardware quirks), and that noisy inputs call for interaction designs where noise is harmless.

PART 16 — GLOSSARY (don't get caught on a term)

- **BLE / GATT:** Bluetooth Low Energy; GATT is its data model of services/characteristics.
- **Characteristic:** a readable/writable/subscribable value on a BLE device.
- **Notification/subscribe:** the device pushes updates to a characteristic.
- **CRC (8/32):** cyclic redundancy check — a checksum to detect corruption.
- **PPG:** photoplethysmography — the optical (green-light) heart-rate sensing.
- **IMU:** inertial measurement unit — accelerometer + gyroscope.
- **R-R interval:** time between heartbeats; basis of HRV.
- **OAuth 2.0:** delegated-authorization standard; *authorization code* (user) vs *client credentials* (app) grants; access + refresh tokens.
- **PKCE:** proof-key extension to OAuth auth-code (guards the code exchange).
- **VCP (Vehicle Command Protocol):** Tesla's signed-command scheme for vehicles.
- **EC / P-256 / prime256v1:** elliptic-curve crypto; the key type Tesla uses for signing.
- **VIN:** vehicle identification number (17 chars).
- **Fail-closed:** on any failure, revert to the safe (stopped) state.
- **Gen4 / Gen5 (5.0/MG):** WHOOP hardware generations; the band here is Gen4 (4.0).

PART 17 — FILE-BY-FILE (what each piece is, so you can point at any file)

File	What it is
whoop_protocol_PWT.py	Protocol core: CRC, build_packet/unframe, command + packet constants, decoders (parse_rt HR, parse_event, decode_historical).
gesture_control_PWT.py	The app: BLE loop, tap debounce, dwell model, safety wiring, haptics, calibrate mode.

File	What it is
vehicle_actuator_PWT.py	Swappable backend: MockActuator, TeslaFleetActuator (raw_command, get_vehicle_data), the command table.
safety_state_machine_PWT.py	Fail-closed FSM, unit-tested.
tesla_auth_PWT.py	OAuth onboarding: register / login / refresh / vehicles.
smart_climate_PWT.py	Body-temp + weather → cabin temp comfort model.
dashboard_PWT.py	Offline HTML dashboard: car state + biometrics.
ble_rawstream_PWT.py / decode_rawstream_PWT.py / plot_rawstream_PWT.py	Raw offload → CSV → chart.
ble_haptic_PWT.py	Buzz-motor test.
launch_PWT.ps1	One-command launcher (token refresh, proxy, controller).
docs/	BUILD_LOG (the journey), QUICKSTART (how to run), this file.

ProjWT — private portfolio project. If you can explain this document, the project is yours. Ask Claude to drill you on any section.